SQL Retail Analytics Project

1. Create Tables

```
CREATE TABLE customers (
customer_id NUMBER,
customer_name VARCHAR2(50)
);

CREATE TABLE stores (
store_id NUMBER,
store_name VARCHAR2(50)
);

CREATE TABLE products (
product_id NUMBER,
category VARCHAR2(50)
);

CREATE TABLE sales (
sale_id NUMBER,
customer_id NUMBER,
store_id NUMBER,
product_id NUMBER,
sale_date DATE,
amount NUMBER,
quantity NUMBER,
discount NUMBER
);
```

2. Insert Data (Auto Generate 1000 Rows)

```
-- Customers
INSERT INTO customers
SELECT LEVEL, 'Cust_'||LEVEL FROM dual CONNECT BY LEVEL<=50;

-- Stores
INSERT INTO stores VALUES (1,'CMR');
INSERT INTO stores VALUES (2,'SR');
```

```
INSERT INTO stores VALUES (3,'Lucky');

-- Products
INSERT INTO products VALUES (1,'Electronics');
INSERT INTO products VALUES (2,'Fashion');
INSERT INTO products VALUES (3,'Grocery');
INSERT INTO products VALUES (4,'Home');

-- Sales (1000 rows)
INSERT INTO sales
SELECT
LEVEL,
MOD(LEVEL,50)+1,
MOD(LEVEL,3)+1,
MOD(LEVEL,4)+1,
DATE '2025-01-01' + MOD(LEVEL,90),
ROUND(DBMS_RANDOM.VALUE(100,10000)),
MOD(LEVEL,5)+1,
CASE WHEN MOD(LEVEL,4)=0 THEN NULL ELSE ROUND(DBMS_RANDOM.VALUE(10,500))
END
FROM dual
CONNECT BY LEVEL <= 1000;
```

1. Data Understanding

Manager Requirement:
Give me overall business performance summary for the last quarter.

Questions:

1. Find total number of transactions.
   select count(sale_id) from sales;

Query result | Script output | DBMS output | Explain Plan | SQL history

Download ▾ Execution time: 0.003 seconds

	COUNT(SALE_ID)
1	1000

2. Find total revenue generated.

```
select sum(amount) as revenue from sales;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download ▾ Execution time: 0.002 seconds

	REVENUE
1	4928986

3. Find total unique customers.

```
select count(distinct CUSTOMER_ID) as unique_cust  from customers;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download ▾ Execution time: 0.003 seconds

	UNIQUE_CUST
1	50

4. Find average order value.

```
select sum(amount)/count(*) as average from sales;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download ▾ Execution time: 0.003 seconds

	AVERAGE
1	4928.986

5. Find total quantity sold.

```
select sum(quantity) as quantity_sold from sales;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download ▾ Execution time: 0.003 seconds

	QUANTITY_SOLD
1	3000

6. Find minimum and maximum sale amount.

```
select max(amount) as max_val ,min(amount) as min_val from sales;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.003 seconds

	MAX_VAL	MIN_VAL
1	9996	133

7. Find average discount given.
   select avg(discount) from sales;

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.003 seconds

	AVG(DISCOUNT)
1	253.396

8. Find total sales per store.
   select s.store_name, sum(sa.amount) as rev from stores s join sales sa on s.store_id=sa.store_id group by s.store_name;

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.003 seconds

	AVG(DISCOUNT)
1	253.396

9. Find total sales per category.
   select p.category, sum(sa.amount) as rev from products p join sales sa on p.PRODUCT_ID=sa.product_id group by p.CATEGORY;

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.005 seconds

	STORE_NAME	REV
1	SR	1615624
2	Lucky	1724044
3	CMR	1589318

10. Find number of transactions per day.
    select sale_date,count(*) as daily_tnsn from sales group by sale_date ORDER by sale_date;

Query result | Script output | DBMS output | Explain Plan

Download | Execution time: 0.005 seconds

	CATEGORY	REV
1	Grocery	1214825
2	Home	1195197
3	Electronics	1264471
4	Fashion	1254493

2. Data Cleaning

Manager Requirement:
Data looks inconsistent, fix NULL issues before analysis.

Questions:

11. Replace NULL discount with 0.

```
select sale_id,nvl(discount,0) as rep from sales;
```

Query result | Script output | DBMS output | Explain Plan

Download | Execution time: 0.004 seconds

	SALE_DATE	DAILY_TNSN
14	1/14/2025, 12:00:0(	11
15	1/15/2025, 12:00:0(	11
16	1/16/2025, 12:00:0(	11
17	1/17/2025, 12:00:0(	11
18	1/18/2025, 12:00:0(	11

12. Find number of NULL discounts.

```
select count(*) as null_val from sales where discount is null;
```

Query result | Script output | DBMS output | Explain Plan

Download | Execution time: 0.004 seconds

	NULL_VAL
1	250

13. Calculate net revenue (amount - discount).

```
select sum(amount-nvl(discount,0)) as net_rev from sales;
```

Query result | Script output | DBMS output | Explain Plan

Download ▾ Execution time: 0.005 seconds

	NET_REV
1	4738939

14. Replace NULL quantity with 1.

```
select nvl(quantity,1) from sales;
```

Query result | Script output | DBMS output | Explain P

Download ▾ Execution time: 0.007 seconds

	NVL(QUANTITY,1)
1	1
2	2
3	3

15. Identify rows where amount is NULL.

```
SELECT * FROM sales WHERE amount IS NULL;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download ▾ Execution time: 0.006 seconds

SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT

No items to display.

16. Use COALESCE to handle multiple NULL columns.

```
Select  sale_id, COALESCE(amount, 0)   as amount_cleaned, COALESCE(quantity, 1) as
quantity_cleaned,  COALESCE(discount, 0) as discount_cleaned from sales;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.01 seconds

	SALE_ID	CLEAN_AMT	CLEAN_QNT	CLEAN_DIS
1	610	6253	1	223
2	611	9229	2	125
3	612	4590	3	0
4	613	3977	4	151
5	614	3582	5	481

17. Create a cleaned column for revenue.

select sale_id,nvl(amount,0)- nvl(discount,0) as rev from sales;

Query result | Script output | DBMS output | Explai

Download | Execution time: 0.007 secon

	SALE_ID	REV
1	610	6030
2	611	9104
3	612	4590
4	613	3826

18. Check percentage of missing data.
select count(case when amount is null then 1 END) /count(*) * 100 as per_amt, count(case when quantity is null then 1 END) /count(*) * 100 as per_qnt,count(case when discount is null then 1 END) /count(*) * 100 as per_dis from sales;

Query result | Script output | DBMS output | Explain Plan | SQL histc

Download | Execution time: 0.004 seconds

	PER_AMT	PER_QNT	PER_DIS
1	0	0	25

19. Flag rows with missing values.
select sale_id ,case when amount is null or quantity is null or discount is null then 'Missing' else 'Complete' end as flag from sales;

Query result | Script output | DBMS output | Ex

Download | Execution time: 0.008 se

	SALE_ID	FLAG
1	610	Complete
2	611	Complete
3	612	Missing

20. Prepare dataset for analysis.

    create or replace view sales_cleaned as select sale_id,customer_id,store_id, product_id, sale_date, nvl(amount,0) as amount,nvl(quantity,1) as quantity, nvl(discount,0) as discount, (nvl(amount,0) - nvl(discount,0)) as net_revenue from sales;

3. Filtering (WHERE)

Manager Requirement:
I want specific insights based on conditions.

Questions:

21. Find sales in last 30 days.
    select * from sales where sale_date > sysdate-30;
    select * from sales where sale_date between date '2025-02-01' and date '2025-03-01';

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.101 seconds

SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT

lo items to display.

22. Find sales above 5000.
    select sale_id,amount from sales where amount>5000;

Query result | Script output | DBMS output | Explain Pla

Download | Execution time: 0.007 seconds

	SALE_ID	AMOUNT
1	610	6253
2	611	9229
3	615	7680

23. Find sales between 1000 and 5000.

```
select sale_id,amount from sales where amount between 1000 and 5000;
```

Query result | Script output | DBMS output | Explain Plan

Download | Execution time: 0.007 seconds

	SALE_ID	AMOUNT
1	612	4590
2	613	3977
3	614	3582

24. Find sales from CMR store.

```
select  sale_id,s.STORE_NAME from sales sa join stores s on sa.store_id=s.store_id where
s.store_name='CMR';
```

Query result | Script output | DBMS output | Explain Pla

Download | Execution time: 0.007 seconds

	SALE_ID	STORE_NAME
1	612	CMR
2	615	CMR
3	618	CMR

25. Find Electronics category sales.

```
select  sa.sale_id,p.category from  sales sa join products p  on p.product_id=sa.product_id
where p.category='Electronics';
```

Query result | Script output | DBMS output | Explain I

Download | Execution time: 0.004 seconds

	SALE_ID	CATEGORY
1	612	Electronics
2	616	Electronics
3	620	Electronics

26. Find sales excluding Grocery category.

```
select  sale_id,p.category from  sales sa join products p  on p.product_id=sa.product_id
where p.category not in('Grocery');
```

Query result | Script output | DBMS output | Expl

Download | Execution time: 0.005 seco

	SALE_ID	CATEGORY
1	611	Home
2	612	Electronics
3	613	Fashion

27. Find customers from specific store.

```
SELECT DISTINCT c.customer_name,s.store_name FROM sales sa JOIN customers c ON sa.customer_id = c.customer_id JOIN stores s ON sa.store_id = s.store_id WHERE s.store_name = 'CMR';
```

Query result | Script output | DBMS output | Explain Pl

Download | Execution time: 0.008 seconds

	CUSTOMER_NAME	STORE_NAME
1	Cust_13	CMR
2	Cust_16	CMR
3	Cust_19	CMR

28. Find high discount transactions.

```
select sale_id,quantity from sales where discount>300;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.008 seconds

	SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
1	614	15	3	3	3/16/2025, 12:00:0(	3582	5	481
2	615	16	1	4	3/17/2025, 12:00:0(	7680	1	437
3	618	19	1	3	3/20/2025, 12:00:0(	7997	4	349
4	619	20	2	4	3/21/2025, 12:00:0(	2775	5	324

29. Find low quantity sales.

```
select * from sales where quantity <2;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.007 seconds

	SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
1	610	11	2	3	3/12/2025, 12:00:0(	6253	1	223
2	615	16	1	4	3/17/2025, 12:00:0(	7680	1	437
3	620	21	3	1	3/22/2025, 12:00:0(	476	1	(null)
4	625	26	2	2	3/27/2025, 12:00:0(	3539	1	473

30. Find recent transactions.

```
select * from sales where sale_date between date '2025-03-21' and date '2025-03-30' ;
select * from sales where sale_date =sysdate-10;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download ▾ Execution time: 0.005 seconds

SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT

No items to display.

4. Aggregation (GROUP BY)

Manager Requirement:
Break down performance across business segments.

Questions:

31. Revenue per store.

```
SELECT s.store_name, SUM(sa.amount) AS revenue FROM sales sa JOIN stores s ON
sa.store_id = s.store_id GROUP BY s.store_name ORDER BY revenue DESC;
```

Query result | Script output | DBMS output | Explain Pl

Download ▾ Execution time: 0.005 seconds

	STORE_NAME	REVENUE
1	Lucky	1724044
2	SR	1615624
3	CMR	1589318

32. Revenue per category.

```
SELECT p.category, SUM(s.amount) AS revenue FROM sales s JOIN products p ON
p.product_id = s.product_id GROUP BY p.CATEGORY ORDER BY revenue DESC;
```

Query result | Script output | DBMS output | Explain Pl

Download | Execution time: 0.007 seconds

	CATEGORY	REVENUE
1	Electronics	1264471
2	Fashion	1254493
3	Grocery	1214825
4	Home	1195197

33. Revenue per customer.

```
SELECT c.customer_name, SUM(sa.amount) AS revenue FROM sales sa left JOIN
customers c ON sa.customer_id = c.customer_id GROUP BY c.customer_name ORDER BY
revenue desc ;
```

Query result | Script output | DBMS output | Explain Pl

Download | Execution time: 0.007 seconds

	CUSTOMER_NAME	REVENUE
1	Cust_36	118741
2	Cust_13	117053
3	Cust_16	115908

34. Monthly revenue.

```
SELECT EXTRACT(MONTH FROM sale_date) AS month,SUM(amount) AS revenue FROM
sales GROUP BY  EXTRACT(MONTH FROM sale_date) ORDER BY  month;
```

Query result | Script output | DBMS output | Explain Pl

Download | Execution time: 0.003 seconds

	MONTH	REVENUE
1	1	1739706
2	2	1495448
3	3	1693832

35. Daily revenue.

```
SELECT sale_date, SUM(amount) AS daily_revenue FROM sales GROUP BY sale_date
ORDER BY sale_date;
```

Query result | Script output | DBMS output | Explain F

Download | Execution time: 0.004 seconds

	SALE_DATE	DAILY_REVENUE
1	1/1/2025, 12:00:00	56704
2	1/2/2025, 12:00:00	68421
3	1/3/2025, 12:00:00	72346
4	1/4/2025, 12:00:00	49994

36. Average sales per store.

```
SELECT s.store_name, round(avg(sa.amount),2) AS revenue FROM sales sa JOIN stores s
ON sa.store_id = s.store_id GROUP BY s.store_name ORDER BY revenue DESC;
```

Query result | Script output | DBMS output | Explain Pla

Download | Execution time: 0.005 seconds

	STORE_NAME	REVENUE
1	Lucky	5177.31
2	SR	4837.2
3	CMR	4772.73

37. Total quantity per category.

```
SELECT p.category, SUM(s.quantity) AS quantity_per_category FROM sales s JOIN
products p ON p.product_id = s.product_id GROUP BY p.CATEGORy;
```

Query result | Script output | DBMS output | Explain Pla

Download | Execution time: 0.005 seconds

	CATEGORY	Q_PER_CATEGORY
1	Grocery	750
2	Home	750
3	Electronics	750
4	Fashion	750

38. Total discount per store.

```
SELECT s.store_name, sum(nvl(sa.discount,0)) AS total_discount FROM sales sa JOIN stores s ON sa.store_id = s.store_id GROUP BY s.store_name;
```

Query result | Script output | DBMS output | Explain Plan

Download | Execution time: 0.037 seconds

	STORE_NAME	TOTAL_DISCOUNT
1	SR	66231
2	Lucky	62858
3	CMR	60958

39. Customer-wise total spending.

```
SELECT c.customer_name, SUM(sa.amount) AS revenue FROM sales sa left JOIN customers c ON sa.customer_id = c.customer_id GROUP BY c.customer_name;
```

Query result | Script output | DBMS output | Expla

Download | Execution time: 0.009 seco

	CUSTOMER_NAME	REVENUE
1	Cust_11	105383
2	Cust_12	87415
3	Cust_13	117053
4	Cust_14	92165

40. Store-wise transaction count.

```
SELECT s.store_name, count(*) AS trans_count FROM sales sa JOIN stores s ON sa.store_id = s.store_id GROUP BY s.store_name order by trans_count desc;
```

Query result | Script output | DBMS output | Explain Pl

Download | Execution time: 0.005 seconds

	STORE_NAME	TRANS_COUNT
1	SR	334
2	CMR	333
3	Lucky	333

5. HAVING

Manager Requirement:
Find top performing segments.

Questions:

41. Customers with revenue > 20000.

```
SELECT c.customer_name, SUM(sa.amount) AS revenue FROM sales sa left JOIN
customers c ON sa.customer_id = c.customer_id GROUP BY c.customer_name having
SUM(sa.amount) > 110000 order by revenue DESC;
```

Query result | Script output | DBMS output | Explain F

Download | Execution time: 0.005 seconds

	CUSTOMER_NAME	REVENUE
1	Cust_36	118741
2	Cust_13	117053
3	Cust_16	115908
4	Cust_31	114029

42. Stores with revenue > 100000.

```
SELECT s.store_name, sum(amount) AS revenue  FROM sales sa left JOIN stores s ON
sa.store_id = s.store_id GROUP BY s.store_name having sum(amount) > 100000;
```

Query result | Script output | DBMS output | Explain Pla

Download | Execution time: 0.005 seconds

	STORE_NAME	REVENUE
1	SR	1615624
2	Lucky	1724044
3	CMR	1589318

43. Categories with avg sales > 3000.

```
SELECT p.category, avg(s.amount) AS avg_sales FROM sales s JOIN products p ON
p.product_id = s.product_id GROUP BY p.CATEGORy having avg(s.amount)>3000;
```

Query result | Script output | DBMS output | Explain Pl

Download | Execution time: 0.006 seconds

	CATEGORY	AVG_SALES
1	Grocery	4859.3
2	Home	4780.788
3	Electronics	5057.884
4	Fashion	5017.972

44. Customers with more than 5 transactions.

```
SELECT c.customer_name, count(*) as transactions FROM sales sa  left JOIN customers
c ON sa.customer_id = c.customer_id GROUP BY c.customer_name having count(*)>5;
```

Query result | Script output | DBMS output | Explain Plan | S

Download | Execution time: 0.006 seconds

	CUSTOMER_NAME	TRANSACTIONS
1	Cust_11	20
2	Cust_12	20
3	Cust_13	20
4	Cust_14	20

45. Stores with high discount usage.

```
SELECT s.store_name, sum(nvl(sa.discount,0)) AS total_discount FROM sales sa left JOIN
stores s ON sa.store_id = s.store_id GROUP BY s.store_name having
sum(nvl(sa.discount,0))>62000;
```

Query result | Script output | DBMS output | Explain Pla

Download | Execution time: 0.006 seconds

	STORE_NAME	TOTAL_DISCOUNT
1	SR	66231
2	Lucky	62858

46. Categories with low performance.

```
SELECT p.category,EXTRACT(MONTH FROM s.sale_date) AS month,SUM(s.amount) AS revenue FROM sales s JOIN products p ON p.product_id = s.product_id GROUP BY p.category, EXTRACT(MONTH FROM s.sale_date) HAVING SUM(s.amount) < 1190000 ORDER BY  month;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.006 seconds

	CATEGORY	MONTH	REVENUE
1	Electronics	1	467229
2	Fashion	1	422224
3	Grocery	1	454654
4	Home	1	395599

47. Customers with high avg order value.

```
SELECT c.customer_name, avg(sa.amount) as avg_order_val FROM sales sa left JOIN customers c ON sa.customer_id = c.customer_id GROUP BY c.customer_name having avg(sa.amount)>4000;
```

Query result | Script output | DBMS output | Explain Pl

Download | Execution time: 0.007 seconds

	CUSTOMER_NAME	AVG_ORDER_VAL
1	Cust_11	5269.15
2	Cust_12	4370.75
3	Cust_13	5852.65
4	Cust_14	4608.25

48. Find top 5 customers using HAVING.

```
SELECT c.customer_name, SUM(s.amount) AS total_revenue FROM sales s JOIN
customers c ON s.customer_id = c.customer_id GROUP BY c.customer_name HAVING
SUM(s.amount) > 100000  ORDER BY total_revenue DESC
FETCH FIRST 5 ROWS ONLY;
```

Query result | Script output | DBMS output | Explain Plar

Download ▾ Execution time: 0.007 seconds

	CUSTOMER_NAME	TOTAL_REVENUE
1	Cust_36	118741
2	Cust_13	117053
3	Cust_16	115908
4	Cust_31	114029
5	Cust_2	111832

49. Stores with max transactions.

```
SELECT s.store_name, COUNT(*) AS transactions FROM sales sa JOIN stores s ON
sa.store_id = s.store_id GROUP BY s.store_name ORDER BY transactions DESC FETCH
FIRST 1 ROW ONLY;
```

Query result | Script output | DBMS output | Explain Plan

Download ▾ Execution time: 0.007 seconds

	STORE_NAME	TRANSACTIONS
1	SR	334

50. Categories with high growth.

```
select p.category,
 sum(case when extract(month from s.sale_date) = 1 then s.amount end) as jan_revenue,
sum(case when extract(month from s.sale_date) = 2 then s.amount end) as
feb_revenue,(sum(case when extract(month from s.sale_date) = 1 then s.amount end)
```

```
sum(case when extract(month from s.sale_date) = 2 then s.amount end)) as growth
from sales s
 join products p on p.product_id = s.product_id group by p.category
having
  (sum(case when extract(month from s.sale_date) = 1 then s.amount end)
  sum(case when extract(month from s.sale_date) = 2 then s.amount end)) > 0
order by growth desc;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download ▾ Execution time: 0.007 seconds

	CATEGORY	JAN_REVENUE	FEB_REVENUE	GROWTH
1	Grocery	454654	346237	108417
2	Electronics	467229	385511	81718
3	Home	395599	363423	32176
4	Fashion	422224	400277	21947

6. CASE WHEN

Manager Requirement:
Segment customers for business decisions.

Questions:

51. Classify customers as High/Medium/Low spenders.

```
SELECT c.customer_name,
        SUM(s.amount) AS total_spent,
    CASE
      WHEN SUM(s.amount) >= 100000 THEN 'High Spender'
      WHEN SUM(s.amount) BETWEEN 90000 AND 100000 THEN 'Medium Spender'
      ELSE 'Low Spender'
    END AS spender_type
  FROM sales s
  JOIN customers c ON s.customer_id = c.customer_id GROUP BY c.customer_name;
```

Query result | Script output | DBMS output | Explain Plan | SQL his

Download | Execution time: 0.005 seconds

	CUSTOMER_NAME	TOTAL_SPENT	SPENDER_TYPE
1	Cust_11	105383	High Spender
2	Cust_12	87415	Low Spender
3	Cust_13	117053	High Spender
4	Cust_14	92165	Medium Spender

52. Classify transactions as Big/Small.

```
select sale_id,amount,(case
 when amount>7000 then 'Big' else 'Small' end)
 as transactions from sales;
```

Query result | Script output | DBMS output | Explain Plan | SQL hi

Download | Execution time: 0.009 seconds

	SALE_ID	AMOUNT	TRANSACTIONS
1	610	6253	Small
2	611	9229	Big
3	612	4590	Small

53. Create discount flag.

```
select sale_id,discount,(case when discount>300 then 'Discount Applied' else 'Discount
Not Applied' end) as flag from sales;
```

ry result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.011 seconds

	SALE_ID	DISCOUNT	FLAG
	613	151	Discount Not Applied
	614	481	Discount Applied
	615	437	Discount Applied
	616	(null)	Discount Not Applied

54. Segment stores based on revenue.

```
select sa.store_name,sum(s.amount) as revenue, (case
 when sum(s.amount)>1700000 then 'Tier 1 store'
when sum(s.amount) between 1570000 and 1700000 then 'Tier 2 store'
 else 'tier 3 store' end ) as segment_stores
from sales s  left join stores sa on s.store_id=sa.store_id group by sa.store_name;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download ▾ Execution time: 0.005 seconds

	STORE_NAME	REVENUE	SEGMENT_STORES
1	SR	1615624	Tier 2 store
2	Lucky	1724044	Tier 1 store
3	CMR	1589318	Tier 2 store

55. Categorize customers based on frequency.

```
   select c.customer_name,count(*) as transactions,(case
 when count(*) >15 then 'Frequent customer'
when count(*) between 11 and 15 then 'Moderate customer ' else 'rare' end) as
customer_frequency   from sales s left join customers c on s.customer_id=c.customer_id
group by c.customer_name;
```

esult | Script output | DBMS output | Explain Plan | SQL history

Download ▾ Execution time: 0.006 seconds

	CUSTOMER_NAME	TRANSACTIONS	CUSTOMER_FREQU
	Cust_11	20	Frequent customer
	Cust_12	20	Frequent customer

56. Create sales buckets.

```
select sale_id ,  amount,
case
     when amount < 1000 then 'bucket a: small'
    when amount between 1000 and 5000 then 'bucket b: medium'
     when amount between 5001 and 10000 then 'bucket c: large'
     else 'bucket d: very large'  end as sales_bucket  from sales;
```

result | Script output | DBMS output | Explain Plan | SQL his

Download | Execution time: 0.01 seconds

SALE_ID	AMOUNT	SALES_BUCKET
610	6253	bucket c: large
611	9229	bucket c: large
612	4590	bucket b: medium
613	3977	bucket b: medium

57. Identify premium customers.

```
select c.customer_name, count(*) as transactions,
    case
     when count(*) > 15 then 'premium_customer'
    when count(*) between 10 and 15 then 'regular_customer'
     else 'rare_customer' end as prem_customers  from sales s
left join customers c on s.customer_id = c.customer_id  group by c.customer_name;
```

Query result | Script output | DBMS output | Explain Plan | SQL histor

Download | Execution time: 0.006 seconds

	CUSTOMER_NAME	TRANSACTIONS	PREM_CUSTOMERS
1	Cust_11	20	premium_customer
2	Cust_12	20	premium_customer
3	Cust_13	20	premium_customer

58. Mark high-value transactions.

```
select sale_id,amount,(case when amount>9000 then 'High Value' else 'Low value' end)
as tns from sales ;
```

Query result | Script output | DBMS output | Explain Plan | S

Download | Execution time: 0.008 seconds

	SALE_ID	AMOUNT	TNS
1	610	6253	Low value
2	611	9229	High Value
3	612	4590	Low value

59. Classify categories based on performance.

```
select p.category, sum(s.amount) as revenue,
 case
        when sum(s.amount) >= 1230000 then 'high performing'
        when sum(s.amount) between 1000000 and 1230000 then 'medium performing'
        else 'low performing'
      end as performance   from sales  s join products p on s.product_id = p.product_id
   group by p.category;
```

Query result | Script output | DBMS output | Explain Plan | SQL histo

Download ▾ Execution time: 0.006 seconds

	CATEGORY	REVENUE	PERFORMANCE
1	Grocery	1214825	medium performing
2	Home	1195197	medium performing
3	Electronics	1264471	high performing
4	Fashion	1254493	high performing

60. Create custom labels for reporting.

```
select sale_id,amount,case when amount>9000 then 'first'
        when amount between 5000 and 9000 then 'Second'
            else 'Third' end as tns from sales ;
```

Query result | Script output | DBMS output | Explain Plan | SQI

Download ▾ Execution time: 0.008 seconds

	SALE_ID	AMOUNT	TNS
1	610	6253	Second
2	611	9229	first
3	612	4590	Third

7. JOINS (Multiple Tables)

Manager Requirement:
Combine customer, product, and store data for insights.

Questions:

61. Join sales with customers table.

```
SELECT s.sale_id, s.amount, c.customer_name
FROM sales s
left JOIN customers c ON s.customer_id = c.customer_id;
```

Query result | Script output | DBMS output | Explain Plan | SQL histo

Download ▾ Execution time: 0.009 seconds

	SALE_ID	AMOUNT	CUSTOMER_NAME
1	610	6253	Cust_11
2	611	9229	Cust_12
3	612	4590	Cust_13
4	613	3977	Cust_14

62. Join sales with stores table.

```
SELECT s.sale_id, s.amount, sa.store_name
FROM sales s
left JOIN stores sa ON s.store_id = sa.store_id;
```

result | Script output | DBMS output | Explain Plan | SQL his

Download ▾ Execution time: 0.009 seconds

	SALE_ID	AMOUNT	STORE_NAME
	610	6253	SR
	611	9229	Lucky
	612	4590	CMR
	613	3977	SR

63. Join sales with products table.

```
SELECT s.sale_id, s.amount, p.product_id,p.category
FROM sales s
left JOIN products p ON s.product_id = p.product_id;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.011 seconds

	SALE_ID	AMOUNT	PRODUCT_ID	CATEGORY
1	610	6253	3	Grocery
2	611	9229	4	Home
3	612	4590	1	Electronics
4	613	3977	2	Fashion

64. Find customer name with store name and sales.

```
select c.customer_name,sa.store_name,sum(s.amount) as revenue from sales s left join
customers c on s.customer_id=s.customer_id left join
stores sa on s.store_id=sa.store_id group by c.customer_name,sa.store_name;
```

Query result | Script output | DBMS output | Explain Plan | SQL histo

Download | Execution time: 0.02 seconds

	CUSTOMER_NAME	STORE_NAME	REVENUE
1	Cust_1	SR	1615624
2	Cust_2	SR	1615624
3	Cust_3	SR	1615624
4	Cust_4	SR	1615624

65. Find category-wise revenue using joins.

```
select p.category,sum(s.amount) as revenue from sales s left join products p on
s.product_id=p.product_id group by p.category order by revenue desc;
```

Query result | Script output | DBMS output | Explain Pl

Download | Execution time: 0.005 seconds

	CATEGORY	REVENUE
1	Electronics	1264471
2	Fashion	1254493
3	Grocery	1214825
4	Home	1195197

66. Find store-wise customer count.

```
select s.store_name,count(distinct sa.customer_id) as cc from sales sa left join stores s
on s.store_id=sa.store_id group by s.store_name;
```

Query result | Script output | DBMS output | Explai

Download | Execution time: 0.002 secon

	STORE_NAME	CC
1	SR	50
2	Lucky	50
3	CMR	50

67. Find top product per store.

```
select store_name,  category,  revenue  from (
  select st.store_name,  p.category,  sum(s.amount) as revenue   rank() over (partition by
st.store_name order by sum(s.amount) desc) as rnk  from sales s
     join stores st on s.store_id = st.store_id
     join products p on s.product_id = p.product_id
     group by st.store_name, p.category
) t
where rnk = 1;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.008 seconds

	STORE_NAME	CATEGORY	REVENUE
1	CMR	Fashion	429435
2	Lucky	Electronics	461435
3	SR	Fashion	423552

68. Find customers who purchased from multiple stores.

```
select c.customer_name from sales sa left join customers c on
c.customer_id=sa.customer_id
```

```
group by c.customer_name having count(distinct sa.store_id)>1;
```

Query result | Script output | DBM

Download | Executic

	CUSTOMER_NAME
1	Cust_11
2	Cust_12
3	Cust_13
4	Cust_14

69. Find sales with product category.

```
select sa.sale_id,sa.amount,p.category from sales sa left join products p on
p.product_id=sa.product_id;
```

Query result | Script output | DBMS output | Explain Plan

Download | Execution time: 0.008 seconds

	SALE_ID	AMOUNT	CATEGORY
1	610	6253	Grocery
2	611	9229	Home
3	612	4590	Electronics

70. Find revenue per customer per store.

```
select c.customer_name,st.store_name,sum(s.amount) as revenue from sales s left join
customers c on s.customer_id=c.customer_id
left join stores st on s.store_id=st.store_id group by c.customer_name,st.store_name;
```

Query result | Script output | DBMS output | Explain Plan | SQL histo

Download | Execution time: 0.003 seconds

	CUSTOMER_NAME	STORE_NAME	REVENUE
1	Cust_11	SR	31039
2	Cust_12	Lucky	31280
3	Cust_13	CMR	38707

71. Find customers with no transactions (LEFT JOIN).

```
select c.customer_name from customers c left join sales s on
c.customer_id=s.customer_id where s.sale_id is null;
```

Query result | Script output | DBMS ou

Download | Execution tim

CUSTOMER_NAME

No items to display.

72. Find missing product mappings.

```
select s.sale_id from sales s left join products p  on s.product_id=p.product_id where
p.product_id is null;
```

Query result | Script output | DBMS output | Explain Pl

Download | Execution time: 0.004 seconds

CUSTOMER_NAME

No items to display.

73. Find store performance using joins.

```
SELECT st.store_name, SUM(s.amount) AS total_revenue
FROM sales s JOIN stores st ON s.store_id = st.store_id
GROUP BY st.store_name ORDER BY total_revenue DESC;
```

Query result | Script output | DBMS output | Explain Plan

Download | Execution time: 0.005 seconds

	STORE_NAME	TOTAL_REVENUE
1	Lucky	1724044
2	SR	1615624
3	CMR	1589318

74. Find cross-category purchases.

```
select c.customer_name  from sales s
join customers c on s.customer_id = c.customer_id
join products p on s.product_id = p.product_id
group by c.customer_name  having count(distinct p.category) > 1;
```

Query result Script output DBM

Download Executio

	CUSTOMER_NAME
1	Cust_11
2	Cust_12
3	Cust_13

75. Find total revenue using joins.

```
select st.store_name,  sum(s.amount) as total_revenue   from sales s   join stores st on
s.store_id = st.store_id  group by st.store_name  order by total_revenue desc;
```

Query result Script output DBMS output Explai

Download Execution time: 0.006 secon

	STORE_NAME	TOTAL_REVENUE
1	Lucky	1724044
2	SR	1615624
3	CMR	1589318

8. Window Functions

Manager Requirement:
Provide advanced analytics without losing row-level data.

Questions:

76. Calculate total spend per customer using window function.

```
select customer_id,amount,sum(amount) over(partition by customer_id) as spend from
sales;
```

	CUSTOMER_ID	AMOUNT	SPEND
1	1	4670	111380
2	1	3525	111380

77. Rank customers by revenue.

```
select customer_id,sum(amount) as revenue,rank() over (order by sum(amount) desc)
as rank from sales group by customer_id;
```

	CUSTOMER_ID	REVENUE	RANK
1	36	118741	1
2	13	117053	2
3	16	115908	3

78. Find top customer per store.

```
select *
from (
  select st.store_name,
         c.customer_name,
         sum(s.amount) as rev,
```

```
        rank() over (partition by st.store_name order by sum(s.amount) desc) as rnk
    from sales s
    join customers c on c.customer_id = s.customer_id
    join stores st on st.store_id = s.store_id
    group by st.store_name, c.customer_name
)where rnk = 1;
```

	STORE_NAME	CUSTOMER_NAME	REV	RNK
1	CMR	Cust_26	46327	1
2	Lucky	Cust_13	51432	1
3	SR	Cust_15	46303	1

79. Calculate running total of sales.

```
select sale_date,amount,sum(amount) over (order by sale_date rows between
unbounded preceding and current row) as running_total
from sales;
```

	SALE_DATE	AMOUNT	RUNNING_TOTAL
1	1/1/2025, 12:00:00	7134	7134
2	1/1/2025, 12:00:00	4187	11321
3	1/1/2025, 12:00:00	7247	18568

80. Find previous sale using LAG.

```
select sale_id, sale_date, amount, lag(amount) over (order by sale_date) as previous_sale
from sales;
```

	SALE_ID	SALE_DATE	AMOUNT	PREVIOUS_SALE
1	540	1/1/2025, 12:00:00	7134	*(null)*
2	360	1/1/2025, 12:00:00	4187	7134
3	450	1/1/2025, 12:00:00	7247	4187

81. Find next sale using LEAD.

```
select sale_id, sale_date, amount, lead(amount) over (order by sale_date) as
nextday_sale from sales;
```

	SALE_ID	SALE_DATE	AMOUNT	NEXTDAY_SALE
1	540	1/1/2025, 12:00:00	7134	4187
2	360	1/1/2025, 12:00:00	4187	7247
3	450	1/1/2025, 12:00:00	7247	4857

82. Calculate sales growth.

```
select sale_id,sale_date,amount, lag(amount) over (order by sale_date) as lag,
  amount - lag(amount) over (order by sale_date) as sales_growth from sales;
```

	SALE_ID	SALE_DATE	AMOUNT	LAG	SALES_GROWTH
1	540	1/1/2025, 12:00:00	7134	*(null)*	*(null)*
2	360	1/1/2025, 12:00:00	4187	7134	-2947
3	450	1/1/2025, 12:00:00	7247	4187	3060

83. Find difference between transactions.

```
select sale_id, sale_date, amount, amount - lag(amount) over (order by sale_date) as diff
from sales;
```

	SALE_ID	SALE_DATE	AMOUNT	DIFF
1	540	1/1/2025, 12:00:00	7134	*(null)*
2	360	1/1/2025, 12:00:00	4187	-2947
3	450	1/1/2025, 12:00:00	7247	3060

84. Calculate cumulative revenue.

```
select sale_date,amount,sum(amount) over (order by sale_date rows between
unbounded preceding and current row) as cumulative_revenue
from sales;
```

	SALE_DATE	AMOUNT	CUMULATIVE_REVE
1	1/1/2025, 12:00:00	7134	7134
2	1/1/2025, 12:00:00	4187	11321
3	1/1/2025, 12:00:00	7247	18568

85. Find moving average of sales.

```
select sale_id,sale_date,amount,avg(amount) over (order by sale_date rows between 2
preceding and current row) as moving_avg from sales;
```

	SALE_ID	SALE_DATE	AMOUNT	MOVING_AVG
1	540	1/1/2025, 12:00:00	7134	7134
2	360	1/1/2025, 12:00:00	4187	5660.5

86. Rank stores by revenue

```
 select st.store_name,sum(s.amount) as revenue,rank() over (order by sum(s.amount)
desc) as store_rank  from sales s join stores st on s.store_id = st.store_id
group by st.store_name;
```

	STORE_NAME	REVENUE	STORE_RANK
1	Lucky	1724044	1
2	SR	1615624	2
3	CMR	1589318	3

87. Find second highest sale.

```
select amount from (select amount,dense_rank() over (order by amount desc) as rnk
from sales)where rnk = 2;
```

	AMOUNT
1	9995

88. Partition data by store and analyze.

```
select st.store_name,s.sale_id,s.amount,sum(s.amount) over (partition by st.store_name)
as store_total  from sales s join stores st on s.store_id = st.store_id;
```

	STORE_NAME	SALE_ID	AMOUNT	STORE_TOTAL
1	CMR	573	6530	1589318
2	CMR	576	8333	1589318
3	CMR	579	9418	1589318

89. Find customer ranking per store.

```
select st.store_name,c.customer_name,sum(s.amount) as revenue,
rank() over (partition by st.store_name order by sum(s.amount) desc) as rank_in_store
from sales s  join stores st on s.store_id = st.store_id
join customers c on c.customer_id = s.customer_id
group by st.store_name, c.customer_name;
```

	STORE_NAME	CUSTOMER_NAME	REVENUE	RANK_IN_STORE
1	CMR	Cust_26	46327	1
2	CMR	Cust_37	44353	2
3	CMR	Cust_1	42791	3

90. Calculate frequency using window.

```
select customer_id,sale_id,
count(*) over (partition by customer_id) as transaction_count from sales;
```

	CUSTOMER_ID	SALE_ID	TRANSACTION_COU
1	1	650	20
2	1	700	20
3	1	750	20

9. NTILE / Segmentation

Manager Requirement:
Segment customers for marketing.

Questions:

91. Divide customers into 4 segments.

```
select customer_id,sum(amount) as revenue,ntile(4) over (order by sum(amount) desc)
as segment from sales group by customer_id;
```

	CUSTOMER_ID	REVENUE	SEGMENT
1	36	118741	1
2	13	117053	1
3	16	115908	1

92. Identify top 25% customers.

```
select customer_id,sum(amount) as revenue,ntile(4) over (order by sum(amount) desc) as
quartile,
        case
      when ntile(4) over (order by sum(amount) desc) = 1
         then 'top 25% customer'
       else 'bottom 75% customer'
    end as segment
from sales group by customer_id
```

1	36	118741	1	top 25% customer
2	13	117053	1	top 25% customer
3	16	115908	1	top 25% customer

93. Identify bottom 25% customers.

```
select customer_id, sum(amount) as revenue,ntile(4) over (order by sum(amount) desc) as
quartile,
        case
          when ntile(4) over (order by sum(amount) desc) = 4
           then 'bottom 25% customer'
           else 'top 75% customer'
        end as segment
from sales group by customer_id;
```

	CUSTOMER_ID	REVENUE	QUARTILE	SEGMENT
1	36	118741	1	top 75% customer
2	13	117053	1	top 75% customer
3	16	115908	1	top 75% customer

94. Segment stores into 3 groups.

```
select store_id, sum(amount) as revenue,
   ntile(3) over (order by sum(amount) desc) as store_segment  from sales group by
store_id;
```

	STORE_ID	REVENUE	STORE_SEGMENT
1	3	1724044	1
2	2	1615624	2
3	1	1589318	3

95. Create quartiles based on revenue.

```
select customer_id,sum(amount) as revenue,ntile(4) over (order by sum(amount) desc) as
revenue_quartile  from sales group by customer_id;
```

	CUSTOMER_ID	REVENUE	REVENUE_QUARTIL
1	36	118741	1
2	13	117053	1
3	16	115908	1

96. Segment customers per store.

```
select st.store_name, c.customer_name, sum(s.amount) as revenue,  ntile(4) over (partition
by st.store_name order by sum(s.amount) desc) as segment_in_store
 from sales s join stores st
on s.store_id = st.store_id join customers c on c.customer_id = s.customer_id group by
st.store_name, c.customer_name;
```

	STORE_NAME	CUSTOMER_NAME	REVENUE	SEGMENT_IN_STOR
1	CMR	Cust_26	46327	1
2	CMR	Cust_37	44353	1
3	CMR	Cust_1	42791	1

97. Identify premium segment.

```
    select customer_id,
        sum(amount) as revenue,
        ntile(4) over (order by sum(amount) desc) as quartile,
        case
          when ntile(4) over (order by sum(amount) desc) = 1
          then 'top 25% customer'
          else 'other'
        end as segment
from sales group by customer_id;
```

	CUSTOMER_ID	REVENUE	QUARTILE	SEGMENT
1	36	118741	1	top 25% customer
2	13	117053	1	top 25% customer
3	16	115908	1	top 25% customer

98. Find mid-level customers.

```
select customer_id,
        sum(amount) as revenue,
        ntile(4) over (order by sum(amount) desc) as quartile,
        case
          when ntile(4) over (order by sum(amount) desc) in (2,3)
          then 'mid-level customer'
          else 'other'
    end as segment
from sales  group by customer_id;
```

	CUSTOMER_ID	REVENUE	QUARTILE	SEGMENT
1	36	118741	1	other
2	13	117053	1	other
3	16	115908	1	other

99. Analyze bucket distribution.

```
select customer_id,  sum(amount) as revenue,
ntile(5) over (order by sum(amount) desc) as five_dis  from sales  group by customer_id;
```

	CUSTOMER_ID	REVENUE	FIVE_DIS
1	36	118741	1
2	13	117053	1
3	16	115908	1

100. Create marketing segments.

```
select customer_id,
    sum(amount) as revenue,
    case
      when ntile(4) over (order by sum(amount) desc) = 1 then 'premium'
      when ntile(4) over (order by sum(amount) desc) = 2 then 'high'
      when ntile(4) over (order by sum(amount) desc) = 3 then 'medium'
      else 'low'
    end as marketing_segment
from sales
group by customer_id;
```

	CUSTOMER_ID	REVENUE	MARKETING_SEGMI
1	36	118741	premium
2	13	117053	premium
3	16	115908	premium

10. FIRST_VALUE / LAST_VALUE

Manager Requirement:
Compare performance with best and worst.

Questions:

101. Find highest sale.

```
select first_value(amount) over(order by amount desc) as high_value from sales;
```

Query result Script output

Download Exec

	HIGH_VALUE
1	9996

102. Find lowest sale.

```
select last_value(amount) over(order by amount desc rows between unbounded
preceding and unbounded following ) as low_value from sales;
```

Download Executic

	LOW_VALUE
1	133

103. Compare each sale with highest.

```
select sale_id,amount,first_value(amount) over(order by amount desc) as
highest_sale,amount-first_value(amount) over(order by amount desc) as compare from
sales;
```

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.009 seconds

	SALE_ID	AMOUNT	HIGHEST_SALE	COMPARE
1	610	6253	9996	-3743
2	611	9229	9996	-767
3	612	4590	9996	-5406

104. Compare each sale with lowest.

```
select sale_id,  amount,
last_value(amount) over (   order by amount desc
 rows between unbounded preceding and unbounded following) as lowest_sale,
 amount - last_value(amount) over (order by amount desc   rows between unbounded
preceding and unbounded following ) as compare from sales;
```

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.011 seconds

	SALE_ID	AMOUNT	LOWEST_SALE	COMPARE
1	610	6253	133	6120
2	611	9229	133	9096
3	612	4590	133	4457

105. Find highest sale per store.

```
select distinct st.store_name,first_value(s.amount) over(partition by st.store_name
order by s.amount desc) as high_sale from sales s join stores st on s.store_id=st.store_id;
```

Query result | Script output | DBMS output | Expla

Download | Execution time: 0.005 seco

	STORE_NAME	HIGH_SALE
1	CMR	9995
2	Lucky	9994
3	SR	9996

106. Find lowest sale per store.

```
select distinct st.store_name,last_value(s.amount) over(partition by st.store_name order
by s.amount desc rows between unbounded preceding and unbounded following) as
low_sale from sales s join stores st on s.store_id=st.store_id;
```

Query result | Script output | DBMS output | Explain

Download | Execution time: 0.004 seconds

	STORE_NAME	LOW_SALE
1	CMR	153
2	Lucky	142
3	SR	133

107. Compare category performance.

```
select p.category,sum(s.amount) as revenue,
    first_value(p.category) over (order by sum(s.amount) desc) as best_category,
    last_value(p.category) over (order by sum(s.amount) desc rows between
unbounded preceding and unbounded following  ) as worst_category from sales s
join products p on s.product_id = p.product_id group by p.category;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download ▾ Execution time: 0.006 seconds

	CATEGORY	REVENUE	BEST_CATEGORY	WORST_CATEGORY
1	Grocery	1214825	Electronics	Home
2	Home	1195197	Electronics	Home
3	Electronics	1264471	Electronics	Home
4	Fashion	1254493	Electronics	Home

108. Find best customer per store.

```
select st.store_name, c.customer_name, sum(s.amount) as revenue,
first_value(c.customer_name) over (partition by st.store_name order by sum(s.amount)
desc) as best_customer  from sales s
join stores st on s.store_id = st.store_id
join customers c on c.customer_id = s.customer_id  group by st.store_name,
c.customer_name;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download ▾ Execution time: 0.008 seconds

	STORE_NAME	CUSTOMER_NAME	REVENUE	BEST_CUSTOMER
1	CMR	Cust_1	42791	Cust_26
2	CMR	Cust_10	32825	Cust_26
3	CMR	Cust_11	41989	Cust_26

109. Find worst performing category.

```
select category,
sum(s.amount) as revenue, last_value(category) over (  order by sum(s.amount) desc
rows between unbounded preceding and unbounded following ) as worst_category
from sales s
join products p on s.product_id = p.product_id group by category;
```

Query result | Script output | DBMS output | Explain Plan | SQL hist

Download ▾ Execution time: 0.005 seconds

	CATEGORY	REVENUE	WORST_CATEGORY
1	Grocery	1214825	Home
2	Home	1195197	Home
3	Electronics	1264471	Home

110. Analyze gap between best and worst.

```
select distinct
    first_value(amount) over (order by amount desc) as best_amount,
    last_value(amount) over (order by amount desc rows between unbounded
preceding and unbounded following  ) as worst_amount,
    (first_value(amount) over (order by amount desc) -last_value(amount) over ( order
by amount desc  rows between unbounded preceding and unbounded following)) as gap
from sales;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download ▾ Execution time: 0.005 seconds

	BEST_AMOUNT	WORST_AMOUNT	GAP
1	9996	133	9863

11. Date Analysis

Manager Requirement:
Analyze time-based trends.

Questions:

111. Find daily sales.

```
select extract(day from sale_date) as day,sum(amount) as daily_sales
    from sales group by extract(day from sale_date) order by day;
```

Query result | Script output | DBMS output | Explain Plan

Download | Execution time: 0.004 seconds

	DAY	DAILY_SALES
1	1	154018
2	2	186296
3	3	188535

112. Find monthly sales.

```
select extract(month from sale_date) as month, extract(day from sale_date) as
day, sum(amount) as monthly_sales  from sales group by extract(month from sale_date),
extract(day from sale_date)
order by month, day;
```

Query result | Script output | DBMS output | Explain Plan | SQL hist

Download | Execution time: 0.006 seconds

	MONTH	DAY	MONTHLY_SALES
1	1	1	56704
2	1	2	68421
3	1	3	72346
4	1	4	49994
5	1	5	59927

113. Find weekend sales.

```
select sale_date, sum(amount) as weekend_sales from sales
 where to_char(sale_date,'dy') in ('sat','sun') group by sale_date  order by sale_date;
```

Query result | Script output | DBMS output | Explain F

Download | Execution time: 0.005 seconds

	SALE_DATE	WEEKEND_SALES
1	1/4/2025, 12:00:00	49994
2	1/5/2025, 12:00:00	59927
3	1/11/2025, 12:00:0(	57486

114. Find first sale date.

```
select min(sale_date) as first_day_sale from sales;
```

Query result | Script output | DBMS

Download | Execution

	FIRST_DAY_SALE
1	1/1/2025, 12:00:00

115. Find last sale date.

```
Select max(sale_date) as last_day_sale from sales;
```

Query result | Script output | DI

Download | Execu

	LAST_DAY_SALE
1	3/31/2025, 12:00:0(

116. Find sales in specific month.

```
select sale_id,sale_date,amount from sales  where extract(year from sale_date) = 2025
 and extract(month from sale_date) = 3  order by sale_date;
```

Query result | Script output | DBMS output | Explain Plan | SQL histor

Download | Execution time: 0.006 seconds

	SALE_ID	SALE_DATE	AMOUNT
1	689	3/1/2025, 12:00:00	4197
2	779	3/1/2025, 12:00:00	2638
3	869	3/1/2025, 12:00:00	4758

117. Find sales growth month-wise.

```
select year, month, monthly_sales, monthly_sales - lag(monthly_sales) over (order by
year, month) as growth  from (
   select extract(year from sale_date) as year, extract(month from sale_date) as month,
   sum(amount) as monthly_sales from sales group by extract(year from sale_date),
extract(month from sale_date)
) t;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download ▾ Execution time: 0.005 seconds

	YEAR	MONTH	MONTHLY_SALES	GROWTH
1	2025	1	1739706	*(null)*
2	2025	2	1495448	-244258
3	2025	3	1693832	198384

118. Find inactive customers (30 days).

```
 select c.customer_name  from customers c  left join sales s on c.customer_id =
s.customer_id  group by c.customer_name  having max(s.sale_date) < current_date -
interval '30' day;
```

Query result | Script output | DBM

Download ▾ Executic

	CUSTOMER_NAME
1	Cust_11
2	Cust_12
3	Cust_13

119. Find repeat customers.

```
select c.customer_name  from sales s  join customers c on s.customer_id = c.customer_id
group by c.customer_name  having count(*) > 1;
```

Query result | Script output | DB

Download | Execut

	CUSTOMER_NAME
1	Cust_11
2	Cust_12
3	Cust_13

120. Find sales trend.

```
select sale_date, sum(amount) as daily_sales  from sales group by sale_date
order by sale_date;
```

Query result | Script output | DBMS output | Explai

Download | Execution time: 0.004 secon

	SALE_DATE	DAILY_SALES
1	1/1/2025, 12:00:00	56704
2	1/2/2025, 12:00:00	68421
3	1/3/2025, 12:00:00	72346